ComEng 2 - Ergänzung

HS 2026 – Prof. Lars Kamm

Autoren:
Mirko Ratti

Version:
1.0.20260929
https://github.com/mirkoratti/HS2026

Inhaltsverzeichnis

14 Arm Cortex-M4 & Timer **21**
14.1 Watchdog Window-Watchdog-Timer (WWDT) . . . 21
14.2 Spezialtimer STM32G4 . . . 21
14.3 M4 vs M0 . . . 21
14.4 Wichtigkeit Standardisierung von ARM-Cortex . . . 21

15 System & Speicher **21**
15.1 Bestandteile . . . 21
15.2 Vorteile des Cortex-M4 NVIC . . . 21
15.3 Infos über den D-Code Systembus . . . 21

16 Architektur & Befehle **21**
16.1 ISA Thumb-2 Befehlssatz . . . 21
16.2 Adressing Modes . . . 21

17 Floating Point Unit (FPU) **21**
17.1 IEEE 754: Addition / Subtraktion . . . 21
17.2 IEEE 754: Multiplikation / Division . . . 21

18 Schnittstellen & Peripherie **21**
18.1 uC-Peripherie . . . 21
18.2 Schieberegister . . . 21
18.3 Taktgeber in einem uP-System . . . 22
18.4 Direct Memory Access (DMA) . . . 22
18.5 UART . . . 22
18.6 SPI . . . 22
18.7 I2C . . . 22

19 Anhang **22**
19.1 Bit-Shifting HEX . . . 22

14 Arm Cortex-M4 & Timer

14.1 Watchdog Window-Watchdog-Timer (WWDT)

- Zur Überwachung der Code-Ausführung. Erzeugt im Fehlerfall einen Reset.
- Dieser Timer muss jeweils vor Ablauf der Timeout-Zeit zurückgesetzt werden.
- Ansonsten wird eine definierte Aktion ausgeführt (z.B. Reset).

14.2 Spezialtimer STM32G4

HRTIM (high resolution)	LPTIM (low power)
RTC	WWDG (eigenen Code überwachen)

14.3 M4 vs M0

- **M4:** schnellere Taktung als M0 ➟High Performance / neuere Technologie
- **M0:** Low Power / Low Cost (alte Technologie)

14.3.1 Cortex-M4 Prozessor

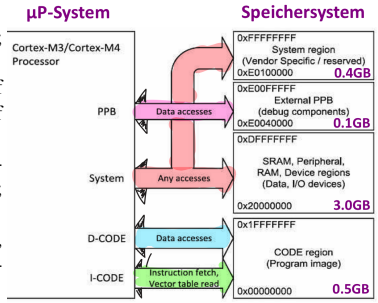
- 3-stufige Pipeline für Thumb-2 Instruktionssatz
- NVIC, SYSTICK & leistungsfähige debug Units
- Memory Protection Unit (MPU) und Floating Point Unit (FPU)
- Erweiterte Busstruktur

  I-Code Bus Instruction- & Vector-Fetch; 32-Bit-Zugriff (Thumb-1).

  D-Code Bus Daten- & Debugzugriff auf ROM (Konstanten); unaligned Zugriff möglich (wird in aligned gewandelt).

  System Bus Zugriff auf SRAM & Peripherie (Variablen); für Code & Daten; unaligned Zugriff möglich.

  PP Bus Privater Peripheriebus (Debug, MPU, FPU); nur word-aligned Transfer.



14.3.2 Vorteile Cortex-M4 gegenüber Cortex-M0

- Floating Point Unit, erweiterte Debug-Unit
- Harvard-Architektur
- Bit-Banding, SIMD-Befehle, erweiterter Befehlssatz, DSP-Funktionen.
- Schnellere Ausführung von Code, unterstützt Thumb-2.
- ALU und Barrel Shifter und Multiply & Accumulate Unit (MAC)

14.4 Wichtigkeit Standardisierung von ARM-Cortex

- Verwendung des gleichen Cores durch verschiedene Chip-Hersteller.
- Es gibt eine Abwärtskompatibilität innerhalb der Familien.

15 System & Speicher

15.1 Bestandteile μC

Hauptbestandteile

- CPU (μP), Speicher (ROM, RAM), Systemkomponenten (Clock).
- Peripherie-Elemente (GPIO, ADC, DAC, Timer, SPI, I2C, . . .)

Grundelemente

- Stromquelle, POR, Taktgeber & Boot-Code (init of internal system)

15.2 Vorteile des Cortex-M4 NVIC

- Kein Software-Overhead für Stacking und Sprünge.
- Parallele Verarbeitung von Code und Daten (Harvard).
- Sehr geringe Jitter-Zeiten bei der Ausführung.
- Nur 12 Cycles für Ein- und Austritt der ESR bzw. ISR.

[Placeholder: Bild Vector-Tabelle / IRQ ISR Handler / Background/Foreground Programm]

15.3 Infos über den D-Code Systembus

- Datenzugriff für ROM (Flash schreiben).
- Debug-Infos aus dem ROM (Code) auslesen.

[Placeholder: Speichertabelle (LA 1-28 Werte)]

16 Architektur & Befehle

16.1 ISA Thumb-2 Befehlssatz

- Verarbeitet 32 und 16 Bit Befehle, Code-Density, Performance, mehr Befehle möglich.
- Flexibler 2. Operand <op2>
- Erweiterte Addressing-Modes
- Erweiterte Grundoperationen (z.B. OR NOT, für jeden 16Bit-Befehl neu auch einen 32Bit-Befehl)
- Bit-Field Processing
- Compare & Branch Zero, Conditional Execution (IT-Instruction)
- Saturation (32Bit)
- Multiply & Accumulate (32Bit MAC)

16.1.1 Single Instruction Multiple Data (SIMD)

- ganze Arrays von Daten werden übergeben.
- Saturation (8/16Bit)
- Multiply & Add/Sub oder Accumulate (8/16Bit)
- Add & Sub (8/16Bit)
- Add/Sub & Saturation

16.1.2 Load/Store-Architektur (RISC)

Sign Extender, Barrel-Shifter (Erlaubt Shift-Operationen auf R).

16.1.3 Multiply & Accumulate (MAC)

Erlaubt Rechenoperationen mit 3 Werten.

16.2 Adressing Modes

Mode	Beispiel
Direkt	`R1` or `#100`
Liste	`(R1-R3, R5, LR)`
Indirekt	`[R0]`
Indirekt Offset	`[R0, #4]`
Indirekt Index	`[R0, R1]`
Indirekt Pre-Index	`[R0, #2]!`
Indirekt Post-Index	`[R0], #-8`
Indirekt Shifted Index	`[R0, R1, LSL #1]`
PC-relative	`label` or `[PC, #4]`
SP-relativ	`[SP, #4]`

[Placeholder: Architektur Diagramm (MAC, RO-R15, Data Bus, Address Bus)]

17 Floating Point Unit (FPU)

- Program Control & Status Register (FPCSR)
- Grundoperationen: Mathematische Befehle, Datentransferbefehle, Konvertierungsbefehle, Compare-Befehl

Register FPU

- 32 Single Precision Register (S0-S31)
- 16 Double Precision Shadow Register (D0-D15)

17.1 IEEE 754: Addition / Subtraktion

Nötige Schritte:

1. Beide Werte müssen normalisiert sein.
2. Exponenten auf grösseren Wert anpassen.
3. Mantissen addieren bzw. subtrahieren.
4. Resultat wenn nötig normalisieren.
5. Umwandlung in IEEE 754 Format.

Beispiel: $0.74 + 0.09 = 0.83$

Runden: Ergebnis auf nächstliegende Zahl runden (round to nearest).

Problematik: Bei extremer Anpassung der Exponenten wird der Rundungsfehler immer massiver.

[Placeholder: Ausführliche Binär-Berechnung (Addition $0.74 + 0.09$)]

17.2 IEEE 754: Multiplikation / Division

Nötige Schritte:

1. Überprüfung auf Werte mit 0.0f
2. Exponenten addieren bzw. subtrahieren
3. Mantissen multiplizieren bzw. dividieren und Vorzeichen anpassen
4. Resultat wenn nötig normalisieren
5. Umwandlung in IEEE 754 Format

Beispiel: $6.625 \cdot (-0.5) = -3.3125$

Runden: Ebenfalls "round to nearest".

[Placeholder: Ausführliche Binär-Berechnung (Multiplikation $6.625 \cdot -0.5$)]

18 Schnittstellen & Peripherie

18.1 μC-Peripherie

- Ankopplung über Systembus des μC's.
- Hardwaremässige Umsetzung.
- Control-, Daten- & Statusregister.
- Ein- bzw. Ausgabe über I/O-Ports.

18.2 Schieberegister

Parallel In Serial Out (PISO): Einfache Verkettung von Flip-Flops. Zusätzliche Multiplexer nötig.

Serial In Parallel Out (SIPO): Einfache Verkettung von Flip-Flops.

[Placeholder: Diagramme zu PISO / SIPO / Peripherie Blockschaltbild]

18.3 Taktgeber in einem μP-System

Wichtigkeit, den Taktgeber richtig zu wählen:
- Für Synchronität, Genauigkeit (Präzision), Strom/Effizienz.
- Clock soll so gewählt sein, dass alle zeitkritischen Anforderungen in der definierten Zeit abgearbeitet werden können.
- Performance vs. Leistungsverbrauch.

18.4 Direct Memory Access (DMA)

- Daten werden nicht mittels Load/Store über CPU transferiert, sondern direkt über den DMA.
- Daten werden nicht bearbeitet oder verändert, sie werden als ganze Pakete transferiert.
- Erst am Ende bzw. nach der Hälfte eines Datenpaket-Transfers wird ein Interrupt generiert.
- Interrupt Stacking & Fetching wird reduziert.

Anwendungsbeispiele:
- *High Performance:* Datentransfer & Verarbeitung erfolgt in grossen Paketen. CPU kann Daten verarbeiten während der DMA transferiert.
- *Ultra Low Power:* Datentransfer & -Verarbeitung erfolgt in grossen Paketen. CPU ist im Standby Modus während der DMA transferiert.

18.5 UART

- Besitzt keinen Clk. (asynchrone Übertragung).
- Ausschliesslich Punkt-zu-Punkt Verbindung (Tx, Rx).
- Full-Duplex Betrieb uneingeschränkt möglich.
- Diverse Treiber (RS-232, RS-422, RS-485) für längere Distanzen.
- Stromsparend (Push-Pull-Ausgänge).

Synchronisation: Mittels des Start-Bits, welches bei jedem Symbol zu Beginn übertragen wird.
UART vs. RS-232: UART: Logik-Pegel 1 = High, 0 = Low.
RS-232: Schnittstellen-Pegel 1 = $-3V \cdots -12V$, 0 = $+3V \cdots +12V$.

18.6 SPI

Synchrone Schnittstelle mit minimalen Spezifikationen.
- Durch Synchronität höhere Übertragungsraten als mit UART möglich.
- Eingeschränkter Full-Duplex Betrieb oder Half-Duplex.
- Leitungen: CS, SCLK, MOSI, MISO.
- Mehrere Slaves am gleichen Interface. Nur kurze Distanzen möglich.
- Stromsparend (Push-Pull-Ausgänge).

SPI: SW Data-Ready bei Subs
- Problematik: Sub kann dem Main kein "Data Readyßenden, Main muss aktiv ein Status-Register abfragen.
- Vorteil: Keine zusätzlichen Leitungen.
- Nachteile: Ständige Abfrage des Status-Registers, erhöhter Datentransfer, kein effizienter Standby-Betrieb möglich.

SPI: HW Data-Ready bei Subs HW-Abfrage: Zusätzliche Interrupt-Leitungen und Data-Ready-Leitung.
- Vorteile: Keine überflüssiger Datentransfer, effizienter Standby-Betrieb möglich.
- Nachteil: Zusätzliche Leitungen zu SPI-Schnittstelle.

18.7 I2C

Synchrone Schnittstelle mit höheren Spezifikationen.
- Half-Duplex Bus Topologie.
- Nicht stromsparend (Open-Drain-Ausgänge mit Widerstand).

19 Anhang

Hex	A	B	C	D	E	F
Dec	10	11	12	13	14	15
Bin	1010	1011	1100	1101	1110	1111

Parallel: Baudrate: $f_{Baud} = \frac{1}{100\mu s} = 10'000$ Baud
Bitrate: $f_{Bit} = \frac{8}{100\mu s} = 80'000$ Bit/s
Seriell: Baudrate: $f_{Baud} = \frac{1}{100\mu s} = 10'000$ Baud
Bitrate: $f_{Bit} = \frac{1}{100\mu s} = 10'000$ Bit/s

Frequenzabweichung:
Damit die Funktion gewährleistet werden kann, darf bei der Übertragung eines Frames die maximale Abweichung höchstens 1 Bitbreite betragen.

$$f_{c,i} = \left(1 \pm \frac{\Delta t_{max}}{t_{max}}\right) * f_{2,i} = \left(1 \pm \frac{1\text{ Bit}}{2 \cdot \text{Framelänge}}\right) * f_{c,i}$$

[Placeholder: Diagramm / Text bezüglich ADC Rauschen und statistischer Verteilung (Messwerte 460.782)]

19.1 Bit-Shifting HEX

Operazione	**Trucco a mente (hex)**
$x \ll 4$	Aggiungi uno `0` in coda
$x \ll 1$	Raddoppia cifra per cifra (da destra), riporto +1 se cifra ≥ 8
$x \ll 5$	$(x \ll 4) \ll 1$: zero in coda, poi raddoppia
$x \gg 4$	Togli l'ultima cifra hex
$x \gg 1$	Dimezza cifra per cifra (da sinistra), se riporto in entrata: +8 al risultato; riporto in uscita = 1 se cifra dispari
$x \gg 5$	$(x \gg 4) \gg 1$: togli ultima cifra, poi dimezza

Tabella dimezzamento (per $\gg 1$):

cifra	metà	riporto generato
0,2,4,6,8,A,C,E (pari)	$d/2$	no
1,3,5,7,9,B,D,F (dispari)	$\lfloor d/2 \rfloor$	sì $\rightarrow +1$